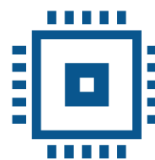




Universitatea *Transilvania* din Brașov
Facultatea de Inginerie Electrică și Știința Calculatoarelor
Departamentul Automatică și Tehnologia Informației



Temă de semestru

Soto Astúa Noelia Inés

Gr. 4LF331, TI

BRAȘOV, 2026

1. Tema proiectului

1.1. Enunț

Proiectul constă în realizarea unei aplicații web numite **Anime Tracker**, destinată gestionării unei colecții personale de anime. Aplicația permite efectuarea operațiilor de bază asupra entității **Anime**: afișarea, adăugarea, modificarea și ștergerea înregistrărilor.

Datele sunt stocate într-o bază de date MySQL, iar aplicația este structurată în două componente principale: frontend, realizat cu React, și backend, realizat cu Spring Boot.

Scopul aplicației este de a oferi o interfață modernă și intuitivă pentru organizarea anime-urilor în funcție de status, gen, progresul de vizionare și alte informații relevante.

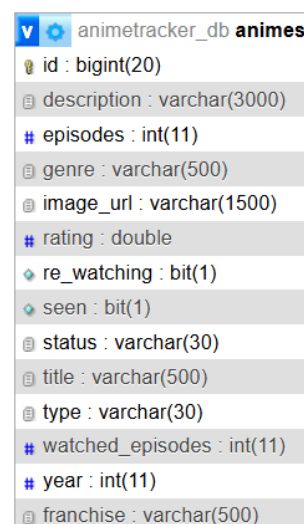
2. Arhitectura aplicației

2.1 Structura bazei de date

Datele aplicației sunt stocate într-o bază de date MySQL numită **animetracker_db**. Baza de date conține tabelul principal **animes**, care păstrează informațiile despre fiecare anime înregistrat în aplicație.

Tabelul include câmpuri pentru identificarea anime-ului, titlu, tip, an, gen, numărul total de episoade, episoadele vizionate, statusul curent, ratingul, imaginea de copertă, descrierea și franciza din care face parte.

Cheia primară a tabelului este câmpul **id**, generat automat, iar celelalte câmpuri sunt folosite pentru afișarea, filtrarea și modificarea datelor în interfața aplicației.



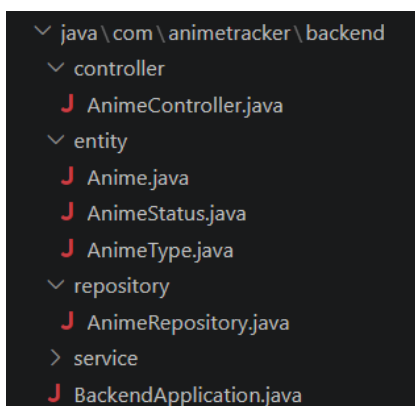
animetracker_db animes	
id	bigint(20)
description	varchar(3000)
episodes	int(11)
genre	varchar(500)
image_url	varchar(1500)
rating	double
re_watching	bit(1)
seen	bit(1)
status	varchar(30)
title	varchar(500)
type	varchar(30)
watched_episodes	int(11)
year	int(11)
franchise	varchar(500)

Figură 1. Structura tabelului *animes* din baza de date *animetracker_db*

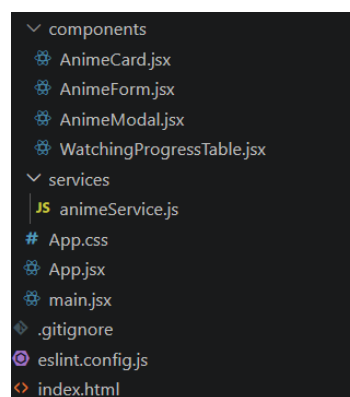
3. Explicații de realizare

Aplicația a fost realizată folosind o arhitectură împărțită în două componente principale: backend și frontend. Backend-ul a fost dezvoltat cu Spring Boot, având rolul de a gestiona logica aplicației, conexiunea cu baza de date și operațiile CRUD asupra entității Anime. Pentru comunicarea cu baza de date MySQL a fost utilizat JPA/Hibernate, iar datele sunt accesate prin intermediul unor endpoint-uri REST.

Frontend-ul a fost realizat cu React, folosind Vite pentru configurarea proiectului. Interfața permite utilizatorului să vizualizeze anime-urile sub formă de carduri, să adauge înregistrări noi, să modifice datele existente și să șteargă anime-uri din colecție. Comunicarea dintre frontend și backend se face prin cereri HTTP, folosind biblioteca Axios.



Figură 2. Structura proiectului backend realizat cu Spring Boot



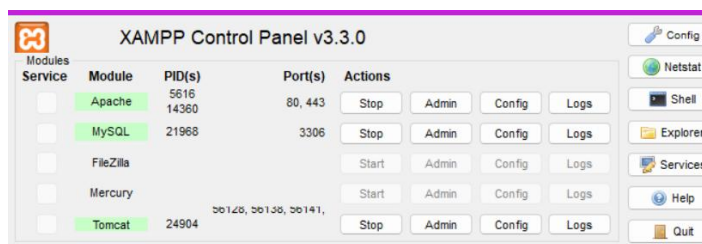
Figură 3. Structura proiectului frontend realizat cu React

În aplicație au fost implementate funcționalități suplimentare precum căutarea după titlu, filtrarea după status și gen, sortarea după titlu, an sau rating, afișarea progresului de vizionare și o fereastră modală pentru detaliile fiecărui anime. De asemenea, aplicația include validări pentru datele introduse, cum ar fi obligativitatea titlului, limitarea ratingului între 0 și 10 și verificarea ca numărul de episoade vizionate să nu fie mai mare decât numărul total de episoade.

4. Manualul de utilizare

4.1 Accesarea aplicației

Pentru ca aplicația să funcționeze corect, trebuie să fie pornite toate componentele proiectului. În primul rând, serverul MySQL trebuie să fie activ, deoarece datele aplicației sunt salvate în baza de date ani metracker_db.



Figură 4. Pornirea serviciilor Apache și MySQL în XAMPP Control Panel

De asemenea, backend-ul realizat cu Spring Boot trebuie să fie pornit, acesta rulând local pe portul 8080.

```
2026-05-31T20:34:49.410+03:00 INFO 23808 --- [Backend] [ restartedMain]
Database JDBC URL [jdbc:mysql://localhost:3306/animetracker_db]
Database driver: MySQL Connector/J
Database dialect: MySQLDialect
Database version: 5.5.5
Default catalog/schema: animetracker_db/undefined
Autocommit mode: undefined/unknown
Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-05-31T20:34:50.238+03:00 INFO 23808 --- [Backend] [ restartedMain]
```

Figură 5. Rularea backend-ului Spring Boot și conectarea la baza de date MySQL

Frontend-ul realizat cu React trebuie pornit separat, folosind comanda `npm run dev`, iar aplicația poate fi accesată din browser la adresa `http://localhost:5173/`. Astfel, frontend-ul trimite cereri către backend, iar backend-ul comunică mai departe cu baza de date.

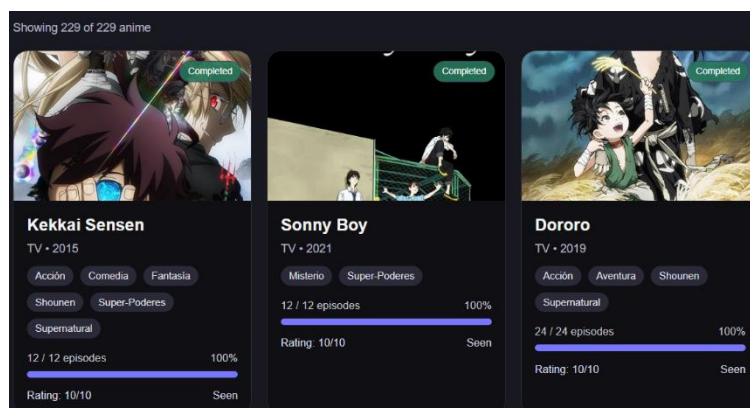
```
VITE v8.0.14 ready in 281 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
8:01:14 p. m. [vite] (client) page reload src/main.jsx
8:01:34 p. m. [vite] (client) hmr update /src/index.css
```

Figură 6. Pornirea frontend-ului React cu Vite și adresa locală a aplicației

4.2 Vizualizarea colecției de anime

După accesarea aplicației, utilizatorul poate vizualiza colecția de anime în secțiunea Anime Library. Anime-urile sunt afișate sub formă de carduri, într-o galerie vizuală. Fiecare card conține imaginea de copertă, titlul anime-ului, tipul, anul apariției, genurile, statusul curent, ratingul și progresul de vizionare.



Figură 7. Vizualizarea colecției de anime sub formă de carduri

4.3 Adăugarea unui anime nou

Pentru adăugarea unui anime nou se folosește formularul Add New Anime, aflat în partea stângă a aplicației. Utilizatorul poate completa câmpuri precum titlul, franciza, tipul, anul apariției, genurile, numărul total de episoade, numărul de episoade vizionate, statusul, ratingul, imaginea de copertă și o scurtă descriere.

The image shows a "Add New Anime" form on a dark background. It has two columns of input fields. The left column contains: "Title", "Franchise (e.g. Boku no Hero Academia)", "TV" (a dropdown menu), "Year", "Genres (e.g. Action, Fantasy, Drama)", "Total episodes", and "Watched episodes". The right column contains: "Watching" (a dropdown menu), "Rating", "Image URL", "Short description", and radio buttons for "Seen" and "Re-watching". At the bottom right is a blue "Add Anime" button.

Figură 8. Formularul pentru adăugarea unui anime nou în aplicație

Câmpul pentru titlu este obligatoriu, deoarece fiecare anime trebuie identificat printr-un nume.

Alte câmpuri, precum imaginea, descrierea, franciza sau numărul total de episoade, pot fi lăsate necompletate, în funcție de informațiile disponibile. După completarea formularului, anime-ul este salvat în baza de date și apare automat în galerie.

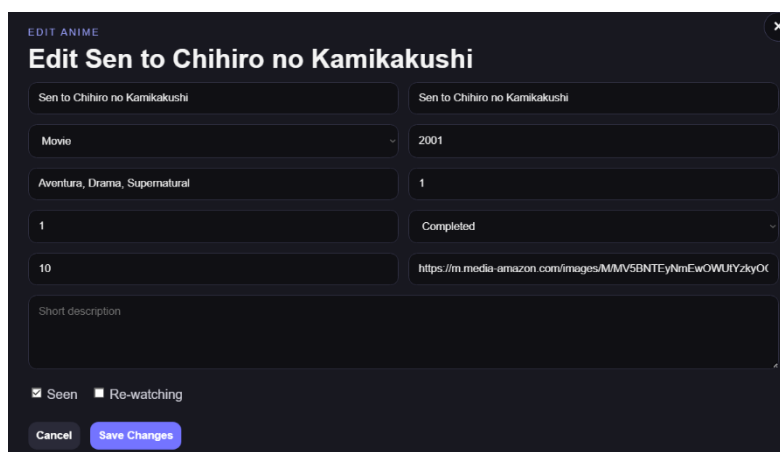
4.4 Vizualizarea detaliilor unui anime

Pentru vizualizarea detaliilor, utilizatorul poate face click pe orice card din galerie. În acel moment se deschide o fereastră de tip pop-up, unde sunt afișate toate informațiile despre anime-ul selectat: imaginea, titlul, tipul, anul, genurile, descrierea, statusul, ratingul, progresul de vizionare și alte detalii relevante.



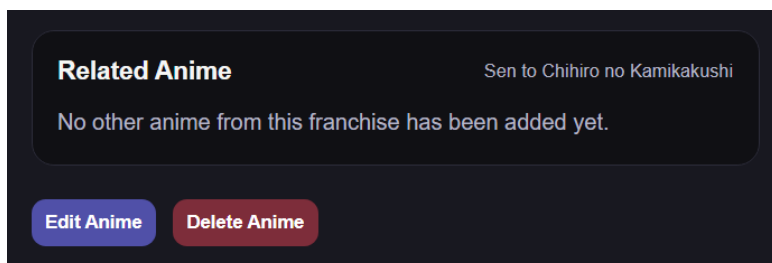
Figură 9. Fereastră de detalii pentru un anime selectat

Din această fereastră, utilizatorul poate apăsa butonul *Edit Anime* pentru a modifica date precum ratingul, statusul, numărul de episoade vizionate, descrierea sau franciza. Tot din aceeași fereastră se poate folosi butonul *Delete Anime*, iar aplicația cere confirmare înainte de ștergerea definitivă a înregistrării.



Figură 10. Formularul de modificare a datelor unui anime

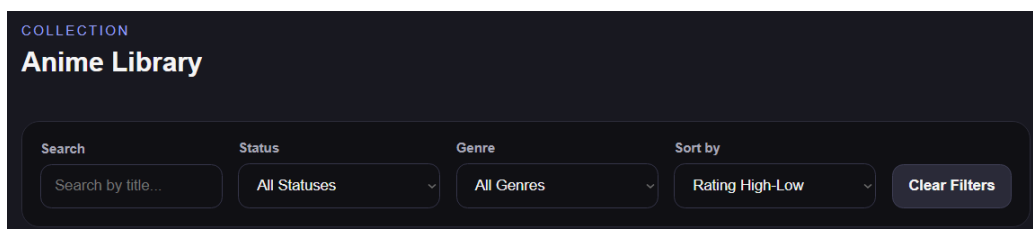
Dacă mai multe anime-uri au aceeași valoare în câmpul Franchise, în fereastra de detalii apare și secțiunea Related Anime. Aceasta afișează alte anime-uri din aceeași franciză, cum ar fi sezoane diferite, filme sau povești secundare. Prin această funcționalitate, utilizatorul poate naviga mai ușor între anime-uri care fac parte din aceeași serie.



Figură 11. Secțiunea pentru anime-uri asociate aceleiași francize

4.5 Căutare, filtrare și sortare

Aplicația include un panou de filtrare în secțiunea Anime Library. Utilizatorul poate căuta anime-uri după titlu, folosind câmpul de căutare. De asemenea, colecția poate fi filtrată după status, de exemplu Watching, Planned, Completed, On Hold sau Dropped. Un alt filtru disponibil este cel după gen, unde utilizatorul poate selecta un gen existent în colecție. În plus, anime-urile pot fi sortate după titlu, an sau rating. Dacă utilizatorul dorește să revină la afișarea inițială, poate folosi butonul Clear Filters, care resetează toate filtrele și sortările aplicate.



Figură 12. Panoul de căutare, filtrare și sortare a colecției de anime

4.6 Actualizarea progresului de vizionare

Pentru anime-urile care au statusul Watching, aplicația afișează o secțiune specială numită Episode Tracker. În această tabelă apar doar anime-urile aflate în curs de vizionare. Utilizatorul poate vedea rapid episodul curent, dacă anime-ul este marcat ca vizionat sau dacă este revizionat. Progresul poate fi actualizat folosind butonul +1, care crește automat numărul de episoade vizionate cu o unitate.

De asemenea, utilizatorul poate introduce manual un număr de episoade și apoi poate apăsa Save. Dacă anime-ul este finalizat, se poate folosi butonul Complete, care schimbă statusul în Completed. În cazul anime-urilor aflate încă în desfășurare, unde numărul total de episoade nu este cunoscut, aplicația afișează progresul ca Ongoing.

CURRENTLY WATCHING

Episode Tracker

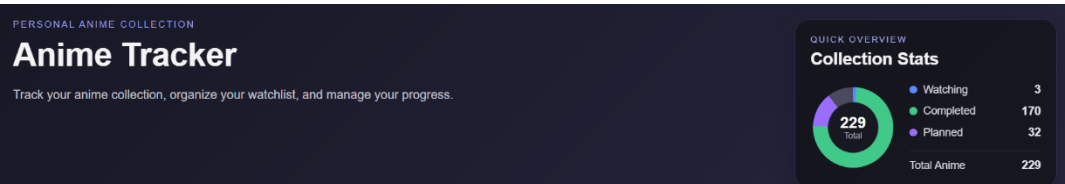
3 active

Re-watching	Seen	Title	Episode	Progress	Actions
☐	☒	One Piece	1019 episodes watched	Ongoing	+1 Save Complete
☐	☒	Tasogare Out Focus	3 / 12	25%	+1 Save Complete
☐	☒	SK8 The Infinity	5 / 12	42%	+1 Save Complete

Figură 13. Tabela Episode Tracker pentru actualizarea progresului de vizionare

4.7 Dashboard-ul aplicației

În partea superioară a aplicației există un dashboard simplu, integrat în bannerul principal. Acesta oferă o privire de ansamblu asupra colecției, afișând numărul total de anime-uri, câte sunt în statusul Watching, câte sunt Completed și câte sunt Planned. Dashboard-ul include și un grafic circular, care reprezintă vizual distribuția anime-urilor în funcție de status. Această secțiune ajută utilizatorul să înțeleagă rapid structura colecției sale fără a analiza manual toate înregistrările.



Figură 14. Dashboard-ul aplicației cu statistici generale și grafic circular

5. Listarea codului sursă integral

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Anime Tracker</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

App.jsx

```
import { useEffect, useMemo, useRef, useState } from "react";
import "./App.css";
import AnimeForm from "./components/AnimeForm";
import AnimeCard from "./components/AnimeCard";
import AnimeModal from "./components/AnimeModal";
import WatchingProgressTable from "./components/WatchingProgressTable";

import {
  createAnime,
  deleteAnime,
  getAllAnimes,
  updateAnime
} from "./services/animeService";

function App() {
  const [animes, setAnimes] = useState([]);
  const [loading, setLoading] = useState(true);
  const [errorMessage, setErrorMessage] = useState("");

  const [searchText, setSearchText] = useState("");
  const [statusFilter, setStatusFilter] = useState("ALL");
  const [genreFilter, setGenreFilter] = useState("ALL");
  const [sortOption, setSortOption] = useState("NONE");

  const [selectedAnime, setSelectedAnime] = useState(null);
  const [modalMode, setModalMode] = useState("details");

  const [toast, setToast] = useState(null);
  const toastTimeoutRef = useRef(null);

  const showToast = (message, type = "success") => {
    setToast({ message, type });

    if (toastTimeoutRef.current) {
      clearTimeout(toastTimeoutRef.current);
    }
  }
}
```



```
    }

    toastTimeoutRef.current = setTimeout(() => {
      setToast(null);
    }, 2600);
  };

  useEffect(() => {
    return () => {
      if (toastTimeoutRef.current) {
        clearTimeout(toastTimeoutRef.current);
      }
    };
  }, []);

  const prepareAnimeForSave = (anime) => {
    const hasKnownTotalEpisodes =
      anime.episodes !== null &&
      anime.episodes !== undefined &&
      Number(anime.episodes) > 0;

    const hasWatchedEpisodes =
      anime.watchedEpisodes !== null &&
      anime.watchedEpisodes !== undefined;

    if (
      hasKnownTotalEpisodes &&
      hasWatchedEpisodes &&
      Number(anime.watchedEpisodes) === Number(anime.episodes)
    ) {
      return {
        ...anime,
        status: "COMPLETED",
        seen: true
      };
    }

    return anime;
  };

  const loadAnimes = async () => {
    try {
      const response = await getAllAnimes();
      setAnimes(response.data);
      setErrorMessage("");
    } catch (error) {
      console.error("Error loading animes:", error);
      setErrorMessage("Could not connect to the backend.");
    }
  };
}
```

```
    } finally {  
      setLoading(false);  
    }  
  };  
  
  useEffect(() => {  
    loadAnimes();  
  }, []);  
  
  const handleCreateAnime = async (anime) => {  
    try {  
      const animeToSave = prepareAnimeForSave(anime);  
      await createAnime(animeToSave);  
      await loadAnimes();  
      showToast("Anime added successfully.");  
    } catch (error) {  
      console.error("Error creating anime:", error);  
      showToast("Could not create anime. Please check the form data.",  
"error");  
    }  
  };  
  
  const handleUpdateAnime = async (id, anime) => {  
    try {  
      const animeToSave = prepareAnimeForSave(anime);  
      await updateAnime(id, animeToSave);  
      await loadAnimes();  
      setSelectedAnime(null);  
      setModalMode("details");  
      showToast("Anime updated successfully.");  
    } catch (error) {  
      console.error("Error updating anime:", error);  
      showToast("Could not update anime. Please check the form data.",  
"error");  
    }  
  };  
  
  const handleQuickUpdateAnime = async (id, anime) => {  
    try {  
      const animeToSave = prepareAnimeForSave(anime);  
      await updateAnime(id, animeToSave);  
      await loadAnimes();  
  
      if (animeToSave.status === "COMPLETED") {  
        showToast("Anime marked as completed.");  
      } else {  
        showToast("Episode progress updated.");  
      }  
    }  
  }  
}
```

```
    } catch (error) {
      console.error("Error updating anime:", error);
      showToast("Could not update anime. Please check the form data.",
"error");
    }
  };

const handleDeleteAnime = async (id) => {
  const confirmDelete = window.confirm(
    "Are you sure you want to delete this anime?"
  );

  if (!confirmDelete) {
    return;
  }

  try {
    await deleteAnime(id);
    await loadAnimes();

    if (selectedAnime?.id === id) {
      setSelectedAnime(null);
    }

    showToast("Anime deleted successfully.");
  } catch (error) {
    console.error("Error deleting anime:", error);
    showToast("Could not delete anime.", "error");
  }
};

const openDetails = (anime) => {
  setSelectedAnime(anime);
  setModalMode("details");
};

const closeModal = () => {
  setSelectedAnime(null);
  setModalMode("details");
};

const getAnimeGenres = (anime) => {
  if (!anime.genre) {
    return [];
  }

  return anime.genre
    .split(",")
```

```
.map((genre) => genre.trim())
.filter(Boolean);
};

const allGenres = Array.from(
  new Set(animes.flatMap((anime) => getAnimeGenres(anime)))
).sort((a, b) => a.localeCompare(b));

const watchingAnimes = animes.filter((anime) => anime.status === "WATCHING");

const filteredAnimes = useMemo(() => {
  const filtered = animes.filter((anime) => {
    const matchesTitle = anime.title
      ?.toLowerCase()
      .includes(searchText.toLowerCase());

    const matchesStatus =
      statusFilter === "ALL" || anime.status === statusFilter;

    const animeGenres = getAnimeGenres(anime);

    const matchesGenre =
      genreFilter === "ALL" || animeGenres.includes(genreFilter);

    return matchesTitle && matchesStatus && matchesGenre;
  });

  const sorted = [...filtered];

  sorted.sort((a, b) => {
    if (sortOption === "TITLE_ASC") {
      return (a.title || "").localeCompare(b.title || "");
    }

    if (sortOption === "YEAR_DESC") {
      return (b.year ?? -Infinity) - (a.year ?? -Infinity);
    }

    if (sortOption === "YEAR_ASC") {
      return (a.year ?? Infinity) - (b.year ?? Infinity);
    }

    if (sortOption === "RATING_DESC") {
      return (b.rating ?? -Infinity) - (a.rating ?? -Infinity);
    }

    if (sortOption === "RATING_ASC") {
      return (a.rating ?? Infinity) - (b.rating ?? Infinity);
    }
  });
});
```

```
    }

    return 0;
  });

  return sorted;
}, [animes, searchText, statusFilter, genreFilter, sortOption]);

const clearFilters = () => {
  setSearchText("");
  setStatusFilter("ALL");
  setGenreFilter("ALL");
  setSortOption("NONE");
};

const hasActiveFilters =
  searchText !== "" ||
  statusFilter !== "ALL" ||
  genreFilter !== "ALL" ||
  sortOption !== "NONE";

const stats = useMemo(() => {
  const total = animes.length;
  const watching = animes.filter((anime) => anime.status ===
"WATCHING").length;
  const completed = animes.filter((anime) => anime.status ===
"COMPLETED").length;
  const planned = animes.filter((anime) => anime.status ===
"PLANNED").length;
  const other = Math.max(total - watching - completed - planned, 0);

  return {
    total,
    watching,
    completed,
    planned,
    other
  };
}, [animes]);

const pieStyle = useMemo(() => {
  const total = stats.total || 1;

  const watchingDeg = (stats.watching / total) * 360;
  const completedDeg = (stats.completed / total) * 360;
  const plannedDeg = (stats.planned / total) * 360;
  const otherDeg = (stats.other / total) * 360;
```

```
    return {
      background: `conic-gradient(
        #5b8cff 0deg ${watchingDeg}deg,
        #41c98a ${watchingDeg}deg ${watchingDeg + completedDeg}deg,
        #9b6dff ${watchingDeg + completedDeg}deg ${watchingDeg + completedDeg +
plannedDeg}deg,
        #4a4a5f ${watchingDeg + completedDeg + plannedDeg}deg ${watchingDeg +
completedDeg + plannedDeg + otherDeg}deg
      )`
    };
  }, [stats]));

const normalizeText = (value) => {
  return value ? value.trim().toLowerCase() : "";
};

const relatedAnimes = selectedAnime
  ? animes.filter((anime) => {
      const selectedFranchise = normalizeText(selectedAnime.franchise);
      const animeFranchise = normalizeText(anime.franchise);

      return (
        anime.id !== selectedAnime.id &&
        selectedFranchise !== "" &&
        animeFranchise === selectedFranchise
      );
    })
  : [];

const openRelatedAnime = (anime) => {
  setSelectedAnime(anime);
  setModalMode("details");
};

return (
  <div className="app">
    {toast && (
      <div className={`toast-message ${toast.type}`}>
        {toast.message}
      </div>
    )}

    <header className="header">
      <div className="header-content">
        <div className="hero-copy">
          <p className="eyebrow">Personal Anime Collection</p>
          <h1>Anime Tracker</h1>
          <p className="subtitle">
```

Track your anime collection, organize your watchlist, and manage your progress.

```
</p>
</div>

<aside className="summary-card">
  <div className="summary-header">
    <div>
      <p className="summary-eyebrow">Quick Overview</p>
      <h3>Collection Stats</h3>
    </div>
  </div>

  <div className="summary-body">
    <div className="summary-pie-wrapper">
      <div className="summary-pie" style={pieStyle}>
        <div className="summary-pie-center">
          <span className="summary-total-number">{stats.total}</span>
          <span className="summary-total-label">Total</span>
        </div>
      </div>
    </div>

    <div className="summary-list">
      <div className="summary-item">
        <span className="summary-label">
          <span className="summary-dot watching-dot"></span>
          Watching
        </span>
        <strong>{stats.watching}</strong>
      </div>

      <div className="summary-item">
        <span className="summary-label">
          <span className="summary-dot completed-dot"></span>
          Completed
        </span>
        <strong>{stats.completed}</strong>
      </div>

      <div className="summary-item">
        <span className="summary-label">
          <span className="summary-dot planned-dot"></span>
          Planned
        </span>
        <strong>{stats.planned}</strong>
      </div>
    </div>
  </div>
</div>
```

```
        <div className="summary-item total-row">
          <span className="summary-label">Total Anime</span>
          <strong>{stats.total}</strong>
        </div>
      </div>
    </div>
  </aside>
</div>
</header>

<main className="main-layout">
  <section>
    <AnimeForm onCreate={handleCreateAnime} />
  </section>

  <section className="content-area">
    <WatchingProgressTable
      animes={watchingAnimes}
      onUpdate={handleQuickUpdateAnime}
    />

    <section className="panel">
      <div className="panel-header">
        <div>
          <p className="section-label">Collection</p>
          <h2>Anime Library</h2>
        </div>
      </div>

      <div className="filters-panel">
        <div className="filter-field search-field">
          <label>Search</label>
          <input
            type="text"
            placeholder="Search by title..."
            value={searchText}
            onChange={(event) => setSearchText(event.target.value)}
          />
        </div>

        <div className="filter-field">
          <label>Status</label>
          <select
            value={statusFilter}
            onChange={(event) => setStatusFilter(event.target.value)}
          >
            <option value="ALL">All Statuses</option>
            <option value="WATCHING">Watching</option>
          </select>
        </div>
      </div>
    </section>
  </section>
</main>
```



```
        <option value="PLANNED">Planned</option>
        <option value="COMPLETED">Completed</option>
        <option value="ON_HOLD">On Hold</option>
        <option value="DROPPED">Dropped</option>
    </select>
</div>

<div className="filter-field">
    <label>Genre</label>
    <select
        value={genreFilter}
        onChange={(event) => setGenreFilter(event.target.value)}
    >
        <option value="ALL">All Genres</option>
        {allGenres.map((genre) => (
            <option key={genre} value={genre}>
                {genre}
            </option>
        ))}
    </select>
</div>

<div className="filter-field">
    <label>Sort by</label>
    <select
        value={sortOption}
        onChange={(event) => setSortOption(event.target.value)}
    >
        <option value="NONE">Default Order</option>
        <option value="TITLE_ASC">Title A-Z</option>
        <option value="YEAR_DESC">Year Newest</option>
        <option value="YEAR_ASC">Year Oldest</option>
        <option value="RATING_DESC">Rating High-Low</option>
        <option value="RATING_ASC">Rating Low-High</option>
    </select>
</div>

<div className="filter-actions">
    <button
        type="button"
        className="clear-filters-button"
        onClick={clearFilters}
        disabled={!hasActiveFilters}
    >
        Clear Filters
    </button>
</div>
</div>
```

```
    {loading && <p>Loading animes...</p>}

    {errorMessage && <p className="error-message">{errorMessage}</p>}

    {!loading && !errorMessage && filteredAnimes.length === 0 && (
      <p className="empty-message">No anime found.</p>
    )}

    {!loading && !errorMessage && filteredAnimes.length > 0 && (
      <>
        <p className="results-count">
          Showing {filteredAnimes.length} of {animes.length} anime
        </p>

        <div className="anime-gallery">
          {filteredAnimes.map((anime) => (
            <AnimeCard
              key={anime.id}
              anime={anime}
              onView={openDetails}
            />
          ))}
        </div>
      </>
    )}
  </section>
</section>
</main>

<AnimeModal
  anime={selectedAnime}
  mode={modalMode}
  onClose={closeModal}
  onChangeMode={setModalMode}
  onUpdate={handleUpdateAnime}
  onDelete={handleDeleteAnime}
  relatedAnimes={relatedAnimes}
  onOpenRelated={openRelatedAnime}
/>
</div>
);
}

export default App;
```

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: #111114;
  color: #f5f5f5;
  font-family: Arial, Helvetica, sans-serif;
}

button,
input,
select,
textarea {
  font-family: inherit;
}

button {
  cursor: pointer;
}

.app {
  min-height: 100vh;
}

.header {
  padding: 48px 64px;
  background: linear-gradient(135deg, #171724, #272740);
  border-bottom: 1px solid #2f2f45;
}

.header-content {
  display: grid;
  grid-template-columns: 1fr 340px;
  gap: 32px;
  align-items: start;
}

.hero-copy {
  min-width: 0;
}

.eyebrow,
.section-label {
  color: #9ca3ff;
  text-transform: uppercase;
  letter-spacing: 2px;
}
```

```
font-size: 12px;
margin: 0 0 8px;
}

.header h1 {
margin: 0;
font-size: 42px;
}

.subtitle {
color: #c7c7d1;
max-width: 650px;
margin-top: 18px;
}

/* Header dashboard */

.summary-card {
background: rgba(16, 16, 20, 0.72);
border: 1px solid #34344c;
border-radius: 20px;
padding: 20px;
backdrop-filter: blur(6px);
box-shadow: 0 10px 30px rgba(0, 0, 0, 0.18);
}

.summary-header {
margin-bottom: 18px;
}

.summary-eyebrow {
color: #a9b3ff;
text-transform: uppercase;
letter-spacing: 2px;
font-size: 11px;
margin: 0 0 6px;
}

.summary-card h3 {
margin: 0;
font-size: 22px;
}

.summary-body {
display: grid;
grid-template-columns: 110px 1fr;
gap: 18px;
align-items: center;
```

```
}

.summary-pie-wrapper {
  display: flex;
  justify-content: center;
}

.summary-pie {
  width: 104px;
  height: 104px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  position: relative;
}

.summary-pie::before {
  content: "";
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: #181820;
  border: 1px solid #303048;
  position: absolute;
}

.summary-pie-center {
  position: relative;
  z-index: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  line-height: 1.1;
}

.summary-total-number {
  font-size: 20px;
  font-weight: 700;
  color: #ffffff;
}

.summary-total-label {
  font-size: 11px;
  color: #b8b8c8;
}

.summary-list {
```

```
    display: grid;
    gap: 10px;
}

.summary-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 12px;
    color: #e6e6f2;
    font-size: 14px;
}

.summary-item strong {
    font-size: 15px;
}

.summary-label {
    display: inline-flex;
    align-items: center;
    gap: 8px;
    color: #d9d9e8;
}

.summary-dot {
    width: 10px;
    height: 10px;
    border-radius: 50%;
    display: inline-block;
}

.watching-dot {
    background: #5b8cff;
}

.completed-dot {
    background: #41c98a;
}

.planned-dot {
    background: #9b6dff;
}

.total-row {
    margin-top: 4px;
    padding-top: 10px;
    border-top: 1px solid #2f2f45;
}
```

```
.main-layout {
  display: grid;
  grid-template-columns: 380px 1fr;
  gap: 24px;
  padding: 32px 64px;
}

.content-area {
  display: grid;
  gap: 24px;
}

.anime-form,
.panel,
.watching-panel {
  background: #181820;
  border: 1px solid #2d2d3a;
  border-radius: 18px;
  padding: 24px;
}

.anime-form h2,
.panel h2,
.watching-panel h2 {
  margin-top: 0;
}

.watching-header,
.panel-header {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  align-items: center;
  margin-bottom: 18px;
}

.panel-subtitle {
  color: #b7b7c7;
  margin-top: 6px;
}

.watching-count {
  background: #2a2a38;
  color: #e8e8f2;
  padding: 7px 12px;
  border-radius: 999px;
  font-size: 13px;
}
```

```
}

.form-grid {
  display: grid;
  gap: 12px;
}

input,
select,
textarea {
  width: 100%;
  background: #101014;
  color: #f5f5f5;
  border: 1px solid #36364a;
  border-radius: 10px;
  padding: 11px 12px;
  outline: none;
}

textarea {
  margin-top: 12px;
  min-height: 90px;
  resize: vertical;
}

input:focus,
select:focus,
textarea:focus {
  border-color: #7777ff;
}

.checkbox-row {
  display: flex;
  gap: 16px;
  margin: 16px 0;
  color: #d3d3dd;
}

.checkbox-row input {
  width: auto;
  margin-right: 6px;
}

.primary-button {
  width: 100%;
  background: #7474ff;
  color: white;
  border: none;
}
```



```
border-radius: 12px;
padding: 12px;
font-weight: bold;
}

.primary-button:hover {
  background: #6262e8;
}

.secondary-button {
  background: #2a2a38;
  color: #f5f5f5;
  border: 1px solid #414158;
  border-radius: 12px;
  padding: 11px 14px;
  font-weight: bold;
}

.secondary-button:hover {
  background: #343447;
}

.toolbar {
  display: grid;
  grid-template-columns: 1fr 220px 220px;
  gap: 16px;
  margin-bottom: 24px;
}

.anime-gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 18px;
}

.anime-card {
  background: #101014;
  border: 1px solid #2d2d3a;
  border-radius: 18px;
  overflow: hidden;
  transition: transform 0.2s ease, border-color 0.2s ease;
}

.anime-card:hover {
  transform: translateY(-4px);
  border-color: #5555aa;
}
```

```
.clickable-card {
  cursor: pointer;
}

.clickable-card:focus {
  outline: 2px solid #7777ff;
  outline-offset: 4px;
}

.anime-cover-wrapper {
  position: relative;
}

.anime-cover {
  width: 100%;
  height: 180px;
  object-fit: cover;
  background: #252532;
  display: block;
}

.card-status,
.modal-status {
  font-size: 11px;
  padding: 6px 10px;
  border-radius: 999px;
  background: rgba(16, 16, 20, 0.85);
  color: white;
  border: 1px solid #454563;
  white-space: nowrap;
}

.card-status {
  position: absolute;
  top: 12px;
  right: 12px;
}

.card-status.watching,
.modal-status.watching {
  background: rgba(49, 91, 143, 0.9);
}

.card-status.planned,
.modal-status.planned {
  background: rgba(103, 81, 143, 0.9);
}
```

```
.card-status.completed,  
.modal-status.completed {  
  background: rgba(40, 120, 93, 0.9);  
}  
  
.card-status.on_hold,  
.modal-status.on_hold {  
  background: rgba(150, 110, 45, 0.9);  
}  
  
.card-status.dropped,  
.modal-status.dropped {  
  background: rgba(130, 45, 60, 0.9);  
}  
  
.anime-card-content {  
  padding: 16px;  
}  
  
.anime-card-content h3 {  
  margin: 0 0 8px;  
  font-size: 20px;  
  line-height: 1.3;  
}  
  
.anime-card-meta {  
  color: #b7b7c7;  
  font-size: 14px;  
  margin: 0 0 12px;  
}  
  
.genre-tags {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 8px;  
  margin-bottom: 16px;  
}  
  
.genre-tag {  
  background: #2a2a38;  
  color: #e8e8f2;  
  padding: 5px 10px;  
  border-radius: 999px;  
  font-size: 12px;  
}  
  
.progress-block {  
  margin-bottom: 14px;
```

```
}

.progress-text {
  display: flex;
  justify-content: space-between;
  color: #c7c7d1;
  font-size: 13px;
  margin-bottom: 8px;
}

.progress-bar {
  height: 8px;
  background: #2a2a38;
  border-radius: 999px;
  overflow: hidden;
}

.progress-fill {
  height: 100%;
  background: #7777ff;
}

.ongoing-progress-bar {
  position: relative;
}

.ongoing-progress-fill {
  width: 100%;
  height: 100%;
  background: linear-gradient(90deg, #5050a8, #7777ff, #5050a8);
  opacity: 0.55;
}

.anime-card-footer {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  color: #dddddd;
  font-size: 13px;
}

.empty-message {
  color: #b7b7c7;
}

.error-message {
  color: #ff8a8a;
}
```

```
/* Watching table */

.watching-table-wrapper {
  overflow-x: auto;
}

.watching-table {
  width: 100%;
  border-collapse: collapse;
  min-width: 760px;
}

.watching-table th,
.watching-table td {
  border-bottom: 1px solid #2d2d3a;
  padding: 12px 10px;
  text-align: left;
  vertical-align: middle;
}

.watching-table th {
  color: #a8a8b8;
  font-weight: 600;
  font-size: 13px;
}

.watching-title-cell {
  font-weight: 700;
  color: #ffffff;
}

.page-icon {
  color: #9ca3ff;
  margin-right: 8px;
}

.episode-control {
  display: flex;
  align-items: center;
  gap: 8px;
}

.episode-control input {
  width: 90px;
  padding: 8px;
}
```

```
.episode-control span {
  color: #b7b7c7;
  white-space: nowrap;
}

.mini-progress {
  display: flex;
  align-items: center;
  gap: 8px;
}

.mini-progress-bar {
  width: 90px;
  height: 7px;
  background: #2a2a38;
  border-radius: 999px;
  overflow: hidden;
}

.mini-progress-fill {
  height: 100%;
  background: #7777ff;
}

.mini-progress span {
  color: #c7c7d1;
  font-size: 13px;
  min-width: 36px;
}

.ongoing-label {
  display: inline-block;
  background: #2a2a38;
  color: #d7d7e2;
  border: 1px solid #414158;
  padding: 6px 10px;
  border-radius: 999px;
  font-size: 12px;
}

.watching-actions {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}

.watching-actions button {
  border: none;
```

```
border-radius: 9px;
padding: 8px 10px;
color: white;
font-weight: bold;
background: #3f3f68;
}

.watching-actions button:hover {
  background: #515184;
}

.watching-actions button:nth-child(2) {
  background: #5050a8;
}

.watching-actions button:nth-child(2):hover {
  background: #6262c8;
}

.watching-actions button:nth-child(3) {
  background: #28785d;
}

.watching-actions button:nth-child(3):hover {
  background: #31926f;
}

/* Modal */

.modal-backdrop {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.72);
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 24px;
  z-index: 1000;
}

.anime-modal {
  position: relative;
  width: min(980px, 100%);
  max-height: 90vh;
  overflow-y: auto;
  background: #181820;
  border: 1px solid #39394d;
  border-radius: 22px;
```

```
    box-shadow: 0 24px 80px rgba(0, 0, 0, 0.55);
    display: grid;
    grid-template-columns: 380px 1fr;
}

.modal-close-button {
    position: absolute;
    top: 14px;
    right: 14px;
    width: 36px;
    height: 36px;
    border: 1px solid #454563;
    background: rgba(16, 16, 20, 0.9);
    color: white;
    border-radius: 50%;
    font-size: 24px;
    line-height: 1;
    z-index: 2;
}

.modal-close-button:hover {
    background: #2a2a38;
}

.modal-cover-section {
    background: #101014;
    border-right: 1px solid #2d2d3a;
}

.modal-cover {
    width: 100%;
    height: 100%;
    min-height: 520px;
    object-fit: cover;
    display: block;
}

.modal-content-section {
    padding: 32px;
}

.modal-title-row {
    display: flex;
    justify-content: space-between;
    gap: 16px;
    align-items: start;
}
```



```
.modal-eyebrow {
  color: #9ca3ff;
  text-transform: uppercase;
  letter-spacing: 2px;
  font-size: 12px;
  margin: 0 0 8px;
}

.modal-content-section h2,
.modal-edit-form h2 {
  margin: 0 0 10px;
  font-size: 32px;
}

.modal-meta {
  color: #b7b7c7;
  margin-bottom: 16px;
}

.modal-description {
  color: #d7d7e2;
  line-height: 1.6;
  margin: 18px 0;
}

.modal-info-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 12px;
  margin: 20px 0;
}

.modal-info-grid div {
  background: #101014;
  border: 1px solid #2d2d3a;
  border-radius: 14px;
  padding: 14px;
}

.info-label {
  display: block;
  color: #9c9caf;
  font-size: 12px;
  margin-bottom: 6px;
}

.modal-progress {
  margin-top: 18px;
}
```

```
}

.modal-actions {
  display: flex;
  gap: 12px;
  margin-top: 24px;
}

.modal-actions button {
  border: none;
  border-radius: 10px;
  padding: 9px 10px;
  color: white;
  font-weight: bold;
}

.modal-actions .primary-button {
  width: auto;
}

.edit-button {
  background: #5050a8;
}

.edit-button:hover {
  background: #6262c8;
}

.delete-button {
  background: #7d2d3a;
}

.delete-button:hover {
  background: #9c3446;
}

.modal-edit-form {
  grid-column: 1 / -1;
  padding: 32px;
}

.modal-form-grid {
  grid-template-columns: repeat(2, 1fr);
}

@media (max-width: 1200px) {
  .header-content {
    grid-template-columns: 1fr;
  }
}
```

```
}  
}  
  
@media (max-width: 1100px) {  
  .toolbar {  
    grid-template-columns: 1fr;  
  }  
}  
  
@media (max-width: 900px) {  
  .main-layout {  
    grid-template-columns: 1fr;  
    padding: 24px;  
  }  
  
  .header {  
    padding: 32px 24px;  
  }  
  
  .anime-modal {  
    grid-template-columns: 1fr;  
  }  
  
  .modal-cover {  
    min-height: auto;  
    height: 300px;  
  }  
  
  .modal-form-grid {  
    grid-template-columns: 1fr;  
  }  
  
  .modal-info-grid {  
    grid-template-columns: 1fr;  
  }  
  
  .summary-body {  
    grid-template-columns: 1fr;  
    justify-items: center;  
  }  
  
  .summary-list {  
    width: 100%;  
  }  
}  
  
/* Filter panel */
```

```
.filters-panel {
  display: grid;
  grid-template-columns: 1fr 180px 180px 180px auto;
  gap: 14px;
  align-items: end;
  background: #101014;
  border: 1px solid #2d2d3a;
  border-radius: 16px;
  padding: 16px;
  margin-bottom: 24px;
}

.filter-field {
  display: grid;
  gap: 7px;
}

.filter-field label {
  color: #a8a8b8;
  font-size: 12px;
  font-weight: 700;
}

.filter-actions {
  display: flex;
  align-items: end;
}

.clear-filters-button {
  background: #2a2a38;
  color: #f5f5f5;
  border: 1px solid #414158;
  border-radius: 10px;
  padding: 11px 14px;
  font-weight: 700;
  white-space: nowrap;
}

.clear-filters-button:hover:not(:disabled) {
  background: #3a3a50;
}

.clear-filters-button:disabled {
  opacity: 0.45;
  cursor: not-allowed;
}

.results-count {
```

```
color: #b7b7c7;
margin: 0 0 18px;
font-size: 14px;
}

/* Form validation */

.field-error {
color: #ff8a8a;
font-size: 12px;
margin: 6px 0 0;
}

/* Toast messages */

.toast-message {
position: fixed;
top: 24px;
right: 24px;
z-index: 2000;
padding: 13px 16px;
border-radius: 12px;
color: white;
font-weight: 700;
box-shadow: 0 12px 35px rgba(0, 0, 0, 0.35);
animation: toastIn 0.2s ease-out;
}

.toast-message.success {
background: #28785d;
}

.toast-message.error {
background: #9c3446;
}

@keyframes toastIn {
from {
opacity: 0;
transform: translateY(-8px);
}

to {
opacity: 1;
transform: translateY(0);
}
}
```

```
@media (max-width: 1250px) {  
  .filters-panel {  
    grid-template-columns: 1fr 1fr;  
  }  
  
  .search-field {  
    grid-column: 1 / -1;  
  }  
  
  .filter-actions {  
    align-items: stretch;  
  }  
  
  .clear-filters-button {  
    width: 100%;  
  }  
}  
  
@media (max-width: 700px) {  
  .filters-panel {  
    grid-template-columns: 1fr;  
  }  
}  
  
/* Related anime section */  
  
.related-section {  
  margin-top: 24px;  
  background: #101014;  
  border: 1px solid #2d2d3a;  
  border-radius: 16px;  
  padding: 18px;  
}  
  
.related-header {  
  display: flex;  
  justify-content: space-between;  
  gap: 12px;  
  align-items: center;  
  margin-bottom: 14px;  
}  
  
.related-header h3 {  
  margin: 0;  
  font-size: 18px;  
}  
  
.related-header span {
```

```
    color: #b7b7c7;
    font-size: 13px;
}

.related-list {
    display: grid;
    gap: 10px;
}

.related-item {
    width: 100%;
    background: #181820;
    border: 1px solid #34344c;
    border-radius: 12px;
    padding: 12px;
    color: #f5f5f5;
    display: flex;
    justify-content: space-between;
    gap: 14px;
    align-items: center;
    text-align: left;
}

.related-item:hover {
    border-color: #7777ff;
    background: #202030;
}

.related-item strong {
    display: block;
    margin-bottom: 4px;
}

.related-item span {
    color: #b7b7c7;
    font-size: 13px;
}

.related-status {
    font-size: 11px;
    padding: 6px 10px;
    border-radius: 999px;
    color: white;
    background: #303048;
    white-space: nowrap;
}

.related-status.watching {
```

```
background: rgba(49, 91, 143, 0.9);
}

.related-status.planned {
  background: rgba(103, 81, 143, 0.9);
}

.related-status.completed {
  background: rgba(40, 120, 93, 0.9);
}

.related-status.on_hold {
  background: rgba(150, 110, 45, 0.9);
}

.related-status.dropped {
  background: rgba(130, 45, 60, 0.9);
}

.related-empty {
  color: #b7b7c7;
  margin: 0;
  line-height: 1.5;
}
```

AnimeCard.jsx

```
function AnimeCard({ anime, onView }) {
  const genreList = anime.genre
    ? anime.genre.split(",").map((item) => item.trim()).filter(Boolean)
    : [];

  const formatStatus = (status) => {
    if (!status) {
      return "Unknown";
    }

    return status
      .toLowerCase()
      .split("_")
      .map((word) => word.charAt(0).toUpperCase() + word.slice(1))
      .join(" ");
  };

  const hasKnownTotalEpisodes =
    anime.episodes !== null && anime.episodes !== undefined && anime.episodes >
    0;
```



```
const progress = hasKnownTotalEpisodes
  ? Math.min(Math.round(((anime.watchedEpisodes ?? 0) / anime.episodes) *
100), 100)
  : null;

const handleKeyDown = (event) => {
  if (event.key === "Enter" || event.key === " ") {
    onView(anime);
  }
};

return (
  <div
    className="anime-card clickable-card"
    onClick={() => onView(anime)}
    onKeyDown={handleKeyDown}
    role="button"
    tabIndex={0}
  >
    <div className="anime-cover-wrapper">
      <img
        src={anime.imageUrl || "https://placeholder.co/400x240?text=No+Image"}
        alt={anime.title}
        className="anime-cover"
      />

      <span className={`card-status ${anime.status?.toLowerCase()}`}>
        {formatStatus(anime.status)}
      </span>
    </div>

    <div className="anime-card-content">
      <h3>{anime.title}</h3>

      <p className="anime-card-meta">
        {anime.type} • {anime.year || "Unknown year"}
      </p>

      <div className="genre-tags">
        {genreList.length > 0 ? (
          genreList.map((genre, index) => (
            <span key={index} className="genre-tag">
              {genre}
            </span>
          ))
        ) : (
          <span className="genre-tag">No genre</span>
        )}
      </div>
    </div>
  </div>
);
```

```
    </div>

    <div className="progress-block">
      <div className="progress-text">
        {hasKnownTotalEpisodes ? (
          <>
            <span>
              {anime.watchedEpisodes ?? 0} / {anime.episodes} episodes
            </span>
            <span>{progress}%</span>
          </>
        ) : (
          <>
            <span>{anime.watchedEpisodes ?? 0} episodes watched</span>
            <span>Ongoing</span>
          </>
        )}
      </div>

      {hasKnownTotalEpisodes ? (
        <div className="progress-bar">
          <div
            className="progress-fill"
            style={{ width: `${progress}%` }}
          ></div>
        </div>
      ) : (
        <div className="progress-bar ongoing-progress-bar">
          <div className="ongoing-progress-fill"></div>
        </div>
      )}
    </div>

    <div className="anime-card-footer">
      <span>Rating: {anime.rating ?? "N/A"}/10</span>
      <span>{anime.seen ? "Seen" : "Not seen"}</span>
    </div>
  </div>
</div>
);
}

export default AnimeCard;
```

AnimeForm.jsx

```
import { useState } from "react";
```

```
const initialFormData = {
  title: "",
  franchise: "",
  type: "TV",
  year: "",
  genre: "",
  episodes: "",
  watchedEpisodes: "",
  status: "WATCHING",
  rating: "",
  seen: false,
  reWatching: false,
  imageUrl: "",
  description: ""
};

function AnimeForm({ onCreate }) {
  const [formData, setFormData] = useState(initialFormData);
  const [formErrors, setFormErrors] = useState({});

  const handleChange = (event) => {
    const { name, value, type, checked } = event.target;

    setFormData({
      ...formData,
      [name]: type === "checkbox" ? checked : value
    });

    setFormErrors({
      ...formErrors,
      [name]: ""
    });
  };

  const toNumberOrNull = (value) => {
    return value === "" ? null : Number(value);
  };

  const validateForm = () => {
    const errors = {};

    if (!formData.title.trim()) {
      errors.title = "Title is required.";
    }

    if (
      formData.rating !== "" &&
      (Number(formData.rating) < 0 || Number(formData.rating) > 10)
    ) {
      errors.rating = "Rating must be between 0 and 10.";
    }
  };
}
```

```
) {
  errors.rating = "Rating must be between 0 and 10.";
}

if (
  formData.episodes !== "" &&
  formData.watchedEpisodes !== "" &&
  Number(formData.watchedEpisodes) > Number(formData.episodes)
) {
  errors.watchedEpisodes =
    "Watched episodes cannot be greater than total episodes.";
}

return errors;
};

const handleSubmit = (event) => {
  event.preventDefault();

  const errors = validateForm();

  if (Object.keys(errors).length > 0) {
    setFormErrors(errors);
    return;
  }

  const animeToSend = {
    ...formData,
    year: toNumberOrNull(formData.year),
    episodes: toNumberOrNull(formData.episodes),
    watchedEpisodes: toNumberOrNull(formData.watchedEpisodes),
    rating: toNumberOrNull(formData.rating)
  };

  onCreate(animeToSend);
  setFormData(initialFormData);
  setFormErrors({});
};

return (
  <form className="anime-form" onSubmit={handleSubmit}>
    <h2>Add New Anime</h2>

    <div className="form-grid">
      <div>
        <input
          name="title"
          placeholder="Title"

```

```
        value={formData.title}
        onChange={handleChange}
      />
      {formErrors.title && (
        <p className="field-error">{formErrors.title}</p>
      )}
    </div>

    <input
      name="franchise"
      placeholder="Franchise (e.g. Boku no Hero Academia)"
      value={formData.franchise}
      onChange={handleChange}
    />

    <select name="type" value={formData.type} onChange={handleChange}>
      <option value="TV">TV</option>
      <option value="MOVIE">Movie</option>
      <option value="OVA">OVA</option>
      <option value="ONA">ONA</option>
      <option value="SPECIAL">Special</option>
    </select>

    <input
      name="year"
      type="number"
      placeholder="Year"
      value={formData.year}
      onChange={handleChange}
    />

    <input
      name="genre"
      placeholder="Genres (e.g. Action, Fantasy, Drama)"
      value={formData.genre}
      onChange={handleChange}
    />

    <input
      name="episodes"
      type="number"
      placeholder="Total episodes"
      value={formData.episodes}
      onChange={handleChange}
    />

    <div>
      <input
```

```
        name="watchedEpisodes"
        type="number"
        placeholder="Watched episodes"
        value={formData.watchedEpisodes}
        onChange={handleChange}
    />
    {formErrors.watchedEpisodes && (
        <p className="field-error">{formErrors.watchedEpisodes}</p>
    )}
</div>

<select name="status" value={formData.status} onChange={handleChange}>
    <option value="WATCHING">Watching</option>
    <option value="PLANNED">Planned</option>
    <option value="COMPLETED">Completed</option>
    <option value="ON_HOLD">On Hold</option>
    <option value="DROPPED">Dropped</option>
</select>

<div>
    <input
        name="rating"
        type="number"
        step="0.1"
        min="0"
        max="10"
        placeholder="Rating"
        value={formData.rating}
        onChange={handleChange}
    />
    {formErrors.rating && (
        <p className="field-error">{formErrors.rating}</p>
    )}
</div>

<input
    name="imageUrl"
    placeholder="Image URL"
    value={formData.imageUrl}
    onChange={handleChange}
/>
</div>

<textarea
    name="description"
    placeholder="Short description"
    value={formData.description}
    onChange={handleChange}
```

```
    />

    <div className="checkbox-row">
      <label>
        <input
          type="checkbox"
          name="seen"
          checked={formData.seen}
          onChange={handleChange}
        />
        Seen
      </label>

      <label>
        <input
          type="checkbox"
          name="reWatching"
          checked={formData.reWatching}
          onChange={handleChange}
        />
        Re-watching
      </label>
    </div>

    <button type="submit" className="primary-button">
      Add Anime
    </button>
  </form>
);
}

export default AnimeForm;
```

AnimeModal.jsx

```
import { useEffect, useState } from "react";

const emptyFormData = {
  title: "",
  franchise: "",
  type: "TV",
  year: "",
  genre: "",
  episodes: "",
  watchedEpisodes: "",
  status: "WATCHING",
  rating: "",
  seen: false,
```

```
    reWatching: false,
    imageUrl: "",
    description: ""
  };

const buildFormData = (anime) => {
  if (!anime) return emptyFormData;

  return {
    title: anime.title || "",
    franchise: anime.franchise || "",
    type: anime.type || "TV",
    year: anime.year ?? "",
    genre: anime.genre || "",
    episodes: anime.episodes ?? "",
    watchedEpisodes: anime.watchedEpisodes ?? "",
    status: anime.status || "WATCHING",
    rating: anime.rating ?? "",
    seen: anime.seen || false,
    reWatching: anime.reWatching || false,
    imageUrl: anime.imageUrl || "",
    description: anime.description || ""
  };
};

function AnimeModal({ anime, mode, onClose, onChangeMode, onUpdate, onDelete,
relatedAnimes = [], onOpenRelated }) {
  const [formData, setFormData] = useState(buildFormData(anime));

  useEffect(() => {
    setFormData(buildFormData(anime));
  }, [anime]);

  if (!anime) {
    return null;
  }

  const genreList = anime.genre
    ? anime.genre.split(",").map((item) => item.trim()).filter(Boolean)
    : [];

  const formatStatus = (status) => {
    if (!status) return "Unknown";

    return status
      .toLowerCase()
      .split("_")
      .map((word) => word.charAt(0).toUpperCase() + word.slice(1))
  }
}
```



```
        .join(" ");
    };

    const progress =
        anime.episodes &&
        anime.episodes > 0 &&
        anime.watchedEpisodes !== null &&
        anime.watchedEpisodes !== undefined
        ? Math.min(Math.round((anime.watchedEpisodes / anime.episodes) * 100),
100)
        : 0;

    const handleChange = (event) => {
        const { name, value, type, checked } = event.target;

        setFormData({
            ...formData,
            [name]: type === "checkbox" ? checked : value
        });
    };

    const toNumberOrNull = (value) => {
        return value === "" ? null : Number(value);
    };

    const handleSubmit = (event) => {
        event.preventDefault();

        if (!formData.title.trim()) {
            alert("Title is required.");
            return;
        }

        if (
            formData.episodes !== "" &&
            formData.watchedEpisodes !== "" &&
            Number(formData.watchedEpisodes) > Number(formData.episodes)
        ) {
            alert("Watched episodes cannot be greater than total episodes.");
            return;
        }

        const updatedAnime = {
            ...formData,
            year: toNumberOrNull(formData.year),
            episodes: toNumberOrNull(formData.episodes),
            watchedEpisodes: toNumberOrNull(formData.watchedEpisodes),
            rating: toNumberOrNull(formData.rating)
        };
    };
}
```

```
};

onUpdate(anime.id, updatedAnime);
};

return (
  <div className="modal-backdrop" onClick={onClose}>
    <div className="anime-modal" onClick={(event) =>
event.stopPropagation()}>
      <button className="modal-close-button" onClick={onClose}>
        ×
      </button>

      {mode === "details" && (
        <>
          <div className="modal-cover-section">
            <img
              src={anime.imageUrl ||
"https://placeholder.co/500x300?text=No+Image"}
              alt={anime.title}
              className="modal-cover"
            />
          </div>

          <div className="modal-content-section">
            <div className="modal-title-row">
              <div>
                <p className="modal-eyebrow">Anime Details</p>
                <h2>{anime.title}</h2>
              </div>

              <span className={`modal-status
${anime.status?.toLowerCase()}`}>
                {formatStatus(anime.status)}
              </span>
            </div>

            <p className="modal-meta">
              {anime.type} • {anime.year || "Unknown year"}
            </p>

            <div className="genre-tags">
              {genreList.length > 0 ? (
                genreList.map((genre, index) => (
                  <span key={index} className="genre-tag">
                    {genre}
                  </span>
                ))
              ) : null}
            </div>
          </div>
        </>
      )}
    </div>
  </div>
);
```

```
    ) : (  
      <span className="genre-tag">No genre</span>  
    )}  
</div>  
  
<p className="modal-description">  
  {anime.description || "No description available."}  
</p>  
  
<div className="modal-info-grid">  
  <div>  
    <span className="info-label">Episodes</span>  
    <strong>  
      {anime.watchedEpisodes ?? 0} / {anime.episodes ?? "?"}  
    </strong>  
  </div>  
  
  <div>  
    <span className="info-label">Progress</span>  
    <strong>{progress}%</strong>  
  </div>  
  
  <div>  
    <span className="info-label">Rating</span>  
    <strong>{anime.rating ?? "N/A"}/10</strong>  
  </div>  
  
  <div>  
    <span className="info-label">Franchise</span>  
    <strong>{anime.franchise || "No franchise"}</strong>  
  </div>  
  
  <div>  
    <span className="info-label">Seen</span>  
    <strong>{anime.seen ? "Yes" : "No"}</strong>  
  </div>  
  
  <div>  
    <span className="info-label">Re-watching</span>  
    <strong>{anime.reWatching ? "Yes" : "No"}</strong>  
  </div>  
</div>  
  
<div className="progress-block modal-progress">  
  <div className="progress-text">  
    <span>Watch progress</span>  
    <span>{progress}%</span>  
  </div>
```

```
<div className="progress-bar">
  <div
    className="progress-fill"
    style={{ width: `${progress}%` }}
  ></div>
</div>

<div className="related-section">
  <div className="related-header">
    <h3>Related Anime</h3>
    <span>{anime.franchise || "No franchise"}</span>
  </div>

  {anime.franchise && relatedAnimes.length > 0 ? (
    <div className="related-list">
      {relatedAnimes.map((relatedAnime) => (
        <button
          key={relatedAnime.id}
          type="button"
          className="related-item"
          onClick={() => onOpenRelated(relatedAnime)}
        >
          <div>
            <strong>{relatedAnime.title}</strong>
            <span>
              {relatedAnime.type} • {relatedAnime.year ||
"Unknown year"}
            </span>
          </div>

            <span className={`related-status
${relatedAnime.status?.toLowerCase()}`}>
              {formatStatus(relatedAnime.status)}
            </span>
          </button>
        </div>
      </div>
    ) : anime.franchise ? (
      <p className="related-empty">
        No other anime from this franchise has been added yet.
      </p>
    ) : (
      <p className="related-empty">
        Add a franchise name to connect this anime with related
seasons, movies or side stories.
      </p>
    )
  )
}
```

```
    })
  </div>

  <div className="modal-actions">
    <button className="edit-button" onClick={() =>
onChangeMode("edit")}>
      Edit Anime
    </button>

    <button className="delete-button" onClick={() =>
onDelete(anime.id)}>
      Delete Anime
    </button>
  </div>
</div>
</>
)}

{mode === "edit" && (
  <form className="modal-edit-form" onSubmit={handleSubmit}>
    <p className="modal-eyebrow">Edit Anime</p>
    <h2>Edit {anime.title}</h2>

    <div className="form-grid modal-form-grid">
      <input
        name="title"
        placeholder="Title"
        value={formData.title}
        onChange={handleChange}
      />

      <input
        name="franchise"
        placeholder="Franchise (e.g. Boku no Hero Academia)"
        value={formData.franchise}
        onChange={handleChange}
      />

      <select name="type" value={formData.type}
onChange={handleChange}>
        <option value="TV">TV</option>
        <option value="MOVIE">Movie</option>
        <option value="OVA">OVA</option>
        <option value="ONA">ONA</option>
        <option value="SPECIAL">Special</option>
      </select>

      <input
```

```
        name="year"
        type="number"
        placeholder="Year"
        value={formData.year}
        onChange={handleChange}
    />

    <input
        name="genre"
        placeholder="Genres (e.g. Action, Fantasy, Drama)"
        value={formData.genre}
        onChange={handleChange}
    />

    <input
        name="episodes"
        type="number"
        placeholder="Total episodes"
        value={formData.episodes}
        onChange={handleChange}
    />

    <input
        name="watchedEpisodes"
        type="number"
        placeholder="Watched episodes"
        value={formData.watchedEpisodes}
        onChange={handleChange}
    />

    <select name="status" value={formData.status}
    onChange={handleChange}>
        <option value="WATCHING">Watching</option>
        <option value="PLANNED">Planned</option>
        <option value="COMPLETED">Completed</option>
        <option value="ON_HOLD">On Hold</option>
        <option value="DROPPED">Dropped</option>
    </select>

    <input
        name="rating"
        type="number"
        step="0.1"
        min="0"
        max="10"
        placeholder="Rating"
        value={formData.rating}
        onChange={handleChange}
```

```
    />

    <input
      name="imageUrl"
      placeholder="Image URL"
      value={formData.imageUrl}
      onChange={handleChange}
    />
  </div>

  <textarea
    name="description"
    placeholder="Short description"
    value={formData.description}
    onChange={handleChange}
  />

  <div className="checkbox-row">
    <label>
      <input
        type="checkbox"
        name="seen"
        checked={formData.seen}
        onChange={handleChange}
      />
      Seen
    </label>

    <label>
      <input
        type="checkbox"
        name="reWatching"
        checked={formData.reWatching}
        onChange={handleChange}
      />
      Re-watching
    </label>
  </div>

  <div className="modal-actions">
    <button
      type="button"
      className="secondary-button"
      onClick={() => onChangeMode("details")}
    >
      Cancel
    </button>
```

```
        <button type="submit" className="primary-button">
          Save Changes
        </button>
      </div>
    </form>
  )}
</div>
</div>
);
}

export default AnimeModal;
```

WatchingProgressTable.jsx

```
import { useState } from "react";

function WatchingProgressTable({ animes, onUpdate }) {
  const [editingEpisodes, setEditingEpisodes] = useState({});

  const getCurrentEpisode = (anime) => {
    if (editingEpisodes[anime.id] !== undefined) {
      return editingEpisodes[anime.id];
    }

    return anime.watchedEpisodes ?? 0;
  };

  const hasKnownTotalEpisodes = (anime) => {
    return anime.episodes !== null && anime.episodes !== undefined &&
anime.episodes > 0;
  };

  const handleEpisodeChange = (animeId, value) => {
    setEditingEpisodes({
      ...editingEpisodes,
      [animeId]: value
    });
  };

  const saveEpisodeProgress = async (anime) => {
    const newValue = Number(getCurrentEpisode(anime));

    if (newValue < 0) {
      alert("Watched episodes cannot be negative.");
      return;
    }
  }
}
```



```
    if (hasKnownTotalEpisodes(anime) && newValue > anime.episodes) {
      alert("Watched episodes cannot be greater than total episodes.");
      return;
    }

    const updatedAnime = {
      ...anime,
      watchedEpisodes: newValue,
      seen: hasKnownTotalEpisodes(anime) ? newValue === anime.episodes :
anime.seen
    };

    await onUpdate(anime.id, updatedAnime);

    const updatedEditingEpisodes = { ...editingEpisodes };
    delete updatedEditingEpisodes[anime.id];
    setEditingEpisodes(updatedEditingEpisodes);
  };

  const increaseEpisode = async (anime) => {
    const currentEpisode = anime.watchedEpisodes ?? 0;
    const nextEpisode = currentEpisode + 1;

    if (hasKnownTotalEpisodes(anime) && nextEpisode > anime.episodes) {
      alert("This anime is already at the last episode.");
      return;
    }

    const updatedAnime = {
      ...anime,
      watchedEpisodes: nextEpisode,
      seen: hasKnownTotalEpisodes(anime) ? nextEpisode === anime.episodes :
anime.seen
    };

    await onUpdate(anime.id, updatedAnime);
  };

  const markAsCompleted = async (anime) => {
    const updatedAnime = {
      ...anime,
      watchedEpisodes: hasKnownTotalEpisodes(anime)
        ? anime.episodes
        : anime.watchedEpisodes,
      status: "COMPLETED",
      seen: true
    };
  };

```

```
    await onUpdate(anime.id, updatedAnime);
  };

  if (animes.length === 0) {
    return (
      <section className="watching-panel">
        <div className="watching-header">
          <div>
            <p className="section-label">Currently Watching</p>
            <h2>Episode Tracker</h2>
          </div>

          <span className="watching-count">0 active</span>
        </div>

        <p className="empty-message">No anime currently marked as Watching.</p>
      </section>
    );
  }

  return (
    <section className="watching-panel">
      <div className="watching-header">
        <div>
          <p className="section-label">Currently Watching</p>
          <h2>Episode Tracker</h2>
        </div>

        <span className="watching-count">{animes.length} active</span>
      </div>

      <div className="watching-table-wrapper">
        <table className="watching-table">
          <thead>
            <tr>
              <th>Re-watching</th>
              <th>Seen</th>
              <th>Title</th>
              <th>Episode</th>
              <th>Progress</th>
              <th>Actions</th>
            </tr>
          </thead>

          <tbody>
            {animes.map((anime) => {
              const currentEpisode = getCurrentEpisode(anime);
              const knownTotal = hasKnownTotalEpisodes(anime);
```

```
const progress = knownTotal
  ? Math.min(
    Math.round(((anime.watchedEpisodes ?? 0) / anime.episodes)
* 100),
    100
  )
  : null;

return (
  <tr key={anime.id}>
    <td>
      <input
        type="checkbox"
        checked={anime.reWatching || false}
        readOnly
      />
    </td>

    <td>
      <input
        type="checkbox"
        checked={anime.seen || false}
        readOnly
      />
    </td>

    <td className="watching-title-cell">
      <span className="page-icon">📺</span>
      {anime.title}
    </td>

    <td>
      <div className="episode-control">
        <input
          type="number"
          min="0"
          max={knownTotal ? anime.episodes : undefined}
          value={currentEpisode}
          onChange={(event) =>
            handleEpisodeChange(anime.id, event.target.value)
          }
        />

        <span>
          {knownTotal ? ` / ${anime.episodes}` : "episodes
watched"}
        </span>
      </div>
    </td>
  </tr>
);
```

```
        </div>
      </td>

      <td>
        {knownTotal ? (
          <div className="mini-progress">
            <div className="mini-progress-bar">
              <div
                className="mini-progress-fill"
                style={{ width: `${progress}%` }}
              ></div>
            </div>
            <span>{progress}%</span>
          </div>
        ) : (
          <span className="ongoing-label">Ongoing</span>
        )}
      </td>

      <td>
        <div className="watching-actions">
          <button onClick={() =>
increaseEpisode(anime)}>+1</button>
          <button onClick={() =>
saveEpisodeProgress(anime)}>Save</button>
          <button onClick={() => markAsCompleted(anime)}>
            Complete
          </button>
        </div>
      </td>
    </tr>
  );
  })}
</tbody>
</table>
</div>
</section>
);
}

export default WatchingProgressTable;
```

Anime.java

```
package com.animetracker.backend.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
```

```
@Entity
@Table(name = "animes")
public class Anime {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Column(length = 500)
    private String title;

    @Column(length = 500)
    private String franchise;

    @Enumerated(EnumType.STRING)
    @NotNull
    private AnimeType type;

    @Min(1900)
    @Max(2100)
    private Integer year;

    @Column(length = 500)
    private String genre;

    @Min(0)
    private Integer episodes;

    @Min(0)
    private Integer watchedEpisodes;

    @Enumerated(EnumType.STRING)
    @NotNull
    private AnimeStatus status;

    @DecimalMin("0.0")
    @DecimalMax("10.0")
    private Double rating;

    private Boolean seen;

    private Boolean reWatching;

    @Column(length = 1500)
    private String imageUrl;
```

```
@Column(length = 3000)
private String description;

public Anime() {
}

public Long getId() {
    return id;
}

public String getTitle() {
    return title;
}

public String getFranchise() {
    return franchise;
}

public void setFranchise(String franchise) {
    this.franchise = franchise;
}

public AnimeType getType() {
    return type;
}

public Integer getYear() {
    return year;
}

public String getGenre() {
    return genre;
}

public Integer getEpisodes() {
    return episodes;
}

public Integer getWatchedEpisodes() {
    return watchedEpisodes;
}

public AnimeStatus getStatus() {
    return status;
}

public Double getRating() {
    return rating;
}
```

```
}

public Boolean getSeen() {
    return seen;
}

public Boolean getReWatching() {
    return reWatching;
}

public String getImageUrl() {
    return imageUrl;
}

public String getDescription() {
    return description;
}

public void setId(Long id) {
    this.id = id;
}

public void setTitle(String title) {
    this.title = title;
}

public void setType(AnimeType type) {
    this.type = type;
}

public void setYear(Integer year) {
    this.year = year;
}

public void setGenre(String genre) {
    this.genre = genre;
}

public void setEpisodes(Integer episodes) {
    this.episodes = episodes;
}

public void setWatchedEpisodes(Integer watchedEpisodes) {
    this.watchedEpisodes = watchedEpisodes;
}

public void setStatus(AnimeStatus status) {
    this.status = status;
}
```

```
}

public void setRating(Double rating) {
    this.rating = rating;
}

public void setSeen(Boolean seen) {
    this.seen = seen;
}

public void setReWatching(Boolean reWatching) {
    this.reWatching = reWatching;
}

public void setImageUrl(String imageUrl) {
    this.imageUrl = imageUrl;
}

public void setDescription(String description) {
    this.description = description;
}
}
```

AnimeStatus.java

```
package com.animetracker.backend.entity;

public enum AnimeStatus {
    WATCHING,
    PLANNED,
    COMPLETED,
    ON_HOLD,
    DROPPED
}
```

AnimeType.java

```
package com.animetracker.backend.entity;

public enum AnimeType {
    TV,
    MOVIE,
    OVA,
    ONA,
    SPECIAL
}
```

AnimeController.java


```
package com.animetracker.backend.controller;

import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.animetracker.backend.entity.Anime;
import com.animetracker.backend.service.AnimeService;

import java.util.List;

@RestController
@RequestMapping("/api/animes")
@CrossOrigin(origins = "http://localhost:5173")
public class AnimeController {

    private final AnimeService animeService;

    public AnimeController(AnimeService animeService) {
        this.animeService = animeService;
    }

    @GetMapping
    public List<Anime> getAllAnimes() {
        return animeService.getAllAnimes();
    }

    @GetMapping("/{id}")
    public Anime getAnimeById(@PathVariable Long id) {
        return animeService.getAnimeById(id);
    }

    @PostMapping
    public ResponseEntity<Anime> createAnime(@Valid @RequestBody Anime anime) {
        Anime createdAnime = animeService.createAnime(anime);
        return new ResponseEntity<>(createdAnime, HttpStatus.CREATED);
    }

    @PutMapping("/{id}")
    public Anime updateAnime(@PathVariable Long id, @Valid @RequestBody Anime anime) {
        return animeService.updateAnime(id, anime);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteAnime(@PathVariable Long id) {
        animeService.deleteAnime(id);
        return ResponseEntity.noContent().build();
    }
}
```

```
}  
}
```

AnimeRepository.java

```
package com.animetracker.backend.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import com.animetracker.backend.entity.Anime;  
  
public interface AnimeRepository extends JpaRepository<Anime, Long> {  
}
```

BackendApplication.java

```
package com.animetracker.backend;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class BackendApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(BackendApplication.class, args);  
    }  
  
}
```